Министерство образования Республики Беларусь

Учреждение образования

«Белорусский государственный университет информатики и радиоэлектроники»

Кафедра электронных вычислительных машин

Лабораторная работа №5
«Программирование клавиатуры»

Выполнил:
Студент группы 450502
Новицкий Е.Ю.

Проверил:
Преподаватель
Марзалюк А.В.

Минск, 2026

1. Постановка задачи

Программируя клавиатуру помигать ее индикаторами. Алгоритм мигания произвольный. Условия реализации программы, необходимые для выполнения лабораторной работы:

1. Запись байтов команды должна выполняться только после проверки незанятости входного регистра контроллера клавиатуры. Проверка осуществляется считывание и анализом регистра состояния контроллера клавиатуры.
2. Для каждого байта команды необходимо считывать и анализировать код возврата. В случае считывания кода возврата, требующего повторить передачу байта, необходимо повторно, при необходимости – несколько раз, выполнить передачу байта. При этом повторная передача данных не исключает выполнения всех оставшихся условий.
3. Для определения момента получения кода возврата необходимо использовать аппаратное прерывания от клавиатуры.

Все коды возврата должны быть выведены на экран в шестнадцатеричной форме.

2. Алгоритм

Для вывода на экран скан-кодов или кодов возврата необходимо заменить обработчик прерывания 09h. При вызове данного обработчика выводится значение из порта 60h на экран. При управлении индикаторами значение из порта 60h (код возврата) необходимо анализировать на случай необходимости повторной передачи байтов команды.

Для управления индикаторами клавиатуры используется команда *EDh*. Второй байт этой команды содержит битовую маску для настройки индикаторов (бит 0 – состояние Scroll Lock, бит 1 – состояние Num Lock, бит 2 – состояние Caps Lock). В данной программе управление индикаторами реализовано в функции void set_mask(byte mask), где mask – битовая маска, определяющая состояние индикаторов.

Перед каждой командой записи происходит ожидание освобождения входного буфера клавиатуры: while((inp(0x64) & 2) != 0);

3. Листинг программы

Далее приведен листинг программы, реализующей все поставленные задачи.

```
#include <stdio.h>
#include <dos.h>
#include <conio.h>

#define KB_DATA_PORT 0x60
#define KB_STATUS_PORT 0x64
```

```
#define KB_CMD_SET_LEDS 0xED
#define KB_ACK_OK 0xFA
#define KB_ACK_RESEND 0xFE

#define PIC1_COMMAND 0x20
#define PIC_EOI 0x20

#define MAX_RETRIES 3
#define ACK_TIMEOUT 60000L

volatile unsigned char irq_code = 0;
volatile int irq_code_ready = 0;
volatile int wait_for_kbd_reply = 0;
volatile unsigned char lastScanCode = 0;
volatile int scanReady = 0;
volatile int capture_scan_code = 0;
volatile int ctrlPressed = 0;

void interrupt (*old_int09_handler)(void);



/* Reads the keyboard controller status byte. */
unsigned char kb_read_status(void);

/* Reads one byte from the keyboard data port. */
unsigned char kb_read_data(void);

/* Sends one byte to the keyboard data port. */
void kb_write_data(unsigned char value);

/* Waits until the keyboard controller input buffer is free. */
void kb_wait_input_free(void);

/* Handles keyboard interrupts while the program waits for replies. */
void interrupt new_int09_handler(void);

/* Installs the custom INT 09h handler. */
void install_int09_handler(void);

/* Restores the original INT 09h handler. */
void restore_int09_handler(void);

/* Waits for ACK or RESEND from the keyboard. */
int wait_for_keyboard_reply(void);

/* Sends a byte to the keyboard and retries on RESEND. */
int send_byte_with_retry(unsigned char value);
```

```
/* Sends the LED command and then the LED mask. */
int set_keyboard_leds(unsigned char mask);

/* Runs the LED blinking demonstration sequence. */
void blink_leds_demo(void);

/* Lets the user enter an LED mask manually. */
void set_mask_manually(void);

/* Waits for a key press and prints its scan code. */
void display_scan_code(void);

/* Prints the main menu. */
void show_menu(void);

/* Starts the main program loop. */
void main(void);



unsigned char kb_read_status(void)
{
    return inp(KB_STATUS_PORT);
}

unsigned char kb_read_data(void)
{
    return inp(KB_DATA_PORT);
}

void kb_write_data(unsigned char value)
{
    outp(KB_DATA_PORT, value);
}

void kb_wait_input_free(void)
{
    while (kb_read_status() & 0x02)
    {
    }
}

void interrupt new_int09_handler(void)
{
    if (wait_for_kbd_reply)
    {
        irq_code = kb_read_data();
```

```
        irq_code_ready = 1;
        outp(PIC1_COMMAND, PIC_EOI);
    }
    else if (capture_scan_code)
    {
        lastScanCode = kb_read_data();

        if (lastScanCode == 0x1D)
        {
            ctrlPressed = 1;
        }
        else if (lastScanCode == 0x9D)
        {
            ctrlPressed = 0;
        }

        scanReady = 1;
        outp(PIC1_COMMAND, PIC_EOI);
    }
    else
    {
        old_int09_handler();
    }
}

void install_int09_handler(void)
{
    disable();
    old_int09_handler = getvect(0x09);
    setvect(0x09, new_int09_handler);
    enable();
}

void restore_int09_handler(void)
{
    disable();
    setvect(0x09, old_int09_handler);
    enable();
}

int wait_for_keyboard_reply(void)
{
    long timeout = ACK_TIMEOUT;

    while (timeout > 0)
    {
        if (irq_code_ready)
        {
```

```
            irq_code_ready = 0;

            /* Ignore unrelated scan codes while waiting for
ACK/RESEND. */
            if (irq_code == KB_ACK_OK || irq_code == KB_ACK_RESEND)
            {
                return (int)irq_code;
            }
        }

        timeout--;
    }

    return -1;
}

int send_byte_with_retry(unsigned char value)
{
    int attempt;
    int reply;

    for (attempt = 1; attempt <= MAX_RETRIES; attempt++)
    {
        irq_code_ready = 0;
        wait_for_kbd_reply = 1;

        /* Wait until the keyboard controller input buffer becomes
free. */
        kb_wait_input_free();
        kb_write_data(value);

        reply = wait_for_keyboard_reply();
        wait_for_kbd_reply = 0;

        if (reply == -1)
        {
            printf("Timeout while waiting for a reply to byte
%02Xh\n", value);
            return 0;
        }

        printf("Keyboard reply: %02Xh\n", reply);

        if (reply == KB_ACK_OK)
        {
            return 1;
        }
```

```
        if (reply == KB_ACK_RESEND)
        {
            printf("Resending byte %02Xh (attempt %d)\n", value,
attempt + 1);
            continue;
        }

        printf("Unexpected reply code: %02Xh\n", reply);
        return 0;
    }

    printf("Error: failed to send byte %02Xh in %d attempts\n", value,
MAX_RETRIES);
    return 0;
}

int set_keyboard_leds(unsigned char mask)
{
    printf("\nSending command EDh\n");
    if (!send_byte_with_retry(KB_CMD_SET_LEDS))
    {
        printf("Failed to send command EDh\n");
        return 0;
    }

    printf("Sending mask %02Xh\n", mask);
    if (!send_byte_with_retry(mask))
    {
        printf("Failed to send mask %02Xh\n", mask);
        return 0;
    }

    return 1;
}

void blink_leds_demo(void)
{
    int cycle;
    int i;
    unsigned char masks[] = {0x00, 0x01, 0x02, 0x04, 0x07, 0x00};

    clrscr();
    printf("========================================\n");
    printf("      KEYBOARD LED BLINKING DEMO\n");
    printf("========================================\n\n");
    printf("Try not to press keys while the program is running,\n");
    printf("so scan codes do not interfere with FA/FE replies.\n\n");
```

```
    for (cycle = 1; cycle <= 5; cycle++)
    {
        printf("\n------ Cycle %d ------\n", cycle);

        for (i = 0; i < 6; i++)
        {
            printf("Setting mask: %02Xh\n", masks[i]);

            if (!set_keyboard_leds(masks[i]))
            {
                printf("\nExecution stopped because of an error.\n");
                printf("Press any key...");
                getch();
                return;
            }

            delay(400);
        }
    }

    printf("\nBlinking finished.\n");
    printf("Press any key...");
    getch();
}

void set_mask_manually(void)
{
    unsigned int mask;

    clrscr();
    printf("========================================\n");
    printf("        MANUAL LED MASK SETUP\n");
    printf("========================================\n\n");

    printf("Mask format:\n");
    printf("bit 2 -> Caps Lock\n");
    printf("bit 1 -> Num Lock\n");
    printf("bit 0 -> Scroll Lock\n\n");

    printf("Examples:\n");
    printf("0 -> all LEDs off\n");
    printf("1 -> Scroll Lock\n");
    printf("2 -> Num Lock\n");
    printf("4 -> Caps Lock\n");
    printf("7 -> all LEDs on\n\n");

    printf("Enter a mask (0..7): ");
    scanf("%u", &mask);
```

```
    if (mask > 7)
    {
        printf("\nError: the mask must be in the range 0..7\n");
        printf("Press any key...");
        getch();
        return;
    }

    if (set_keyboard_leds((unsigned char)mask))
    {
        printf("\nMask was set successfully.\n");
    }
    else
    {
        printf("\nFailed to set the mask.\n");
    }

    printf("Press any key...");
    getch();
}

void display_scan_code(void)
{
    clrscr();
    printf("========================================\n");
    printf("          DISPLAY SCAN CODE\n");
    printf("========================================\n\n");
    printf("Press any key to display its scan code.\n");
    printf("Press Esc or Ctrl+C to return to the menu.\n\n");

    ctrlPressed = 0;

    while (1)
    {
        scanReady = 0;
        capture_scan_code = 1;

        while (!scanReady)
        {
        }

        capture_scan_code = 0;

        printf("Scan code: %02Xh\n", lastScanCode);

        if (lastScanCode == 0x01 || (ctrlPressed && lastScanCode ==
0x2E))
```

```
        {
            break;
        }
    }

    capture_scan_code = 0;
    ctrlPressed = 0;
}

void show_menu(void)
{
    clrscr();
    printf("========================================\n");
    printf(" Lab Work - Keyboard LED Programming\n");
    printf("            via INT 09h\n");
    printf("========================================\n\n");

    printf("1. Blink keyboard LEDs\n");
    printf("2. Set LED mask manually\n");
    printf("3. Display scan code\n");
    printf("0. Exit\n\n");

    printf("Your choice: ");
}

void main(void)
{
    int choice;

    /* Install a temporary INT 09h handler to catch keyboard replies.
*/
    install_int09_handler();

    do
    {
        show_menu();
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                blink_leds_demo();
                break;

            case 2:
                set_mask_manually();
                break;
```

```
            case 3:
                display_scan_code();
                break;

            case 0:
                break;

            default:
                printf("\nInvalid menu item.\n");
                printf("Press any key...");
                getch();
                break;
        }
    } while (choice != 0);

    restore_int09_handler();
}
```

4. Тестирование программы

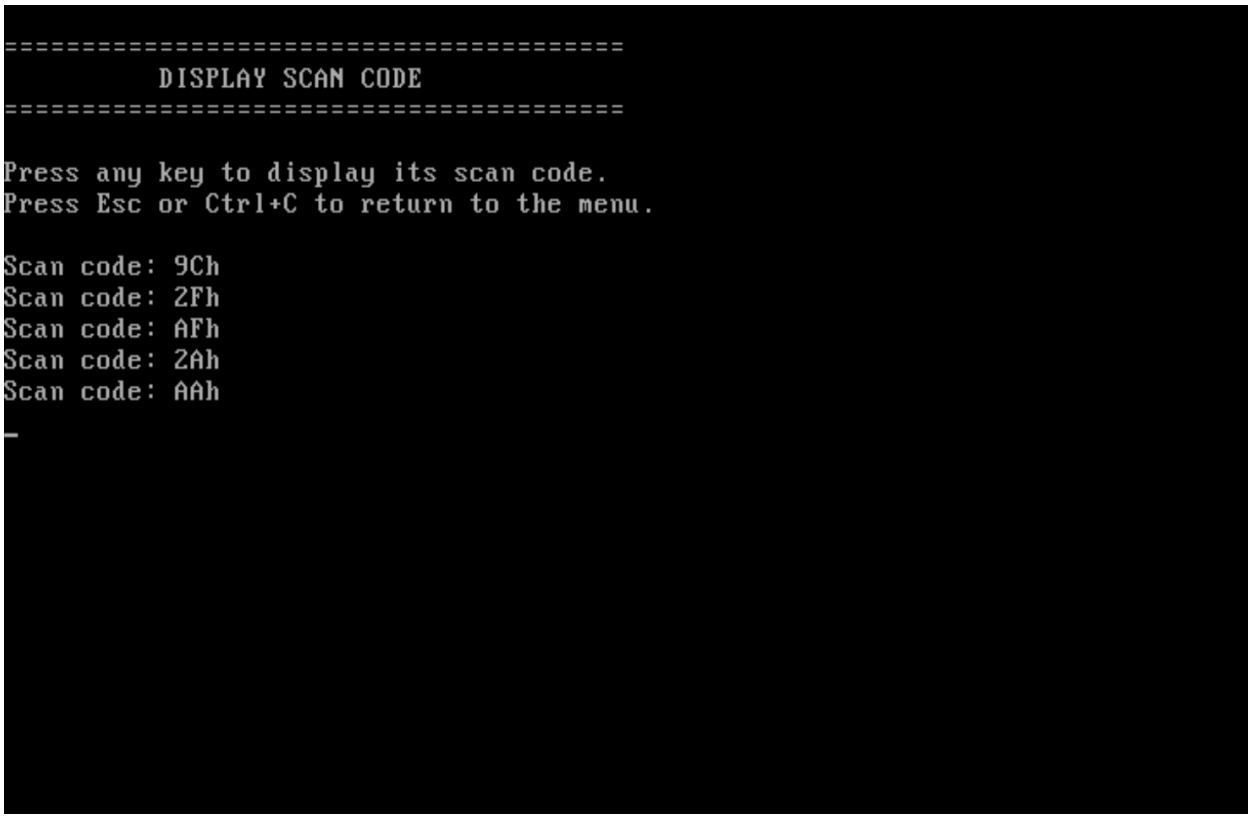

Рисунок 4.1. — Вывод скан-кода при нажатии на клавишу.

5. Заключение

В данной лабораторной работе были выполнены все поставленные задачи: программа выводит на экран скан-коды клавиш при их нажатии и отпускании, также реализовано мигание индикаторов пока не будет нажата клавиша Esc.

Программа компилировалась в Turbo C++ и запускалась в DOSBox-X.